Loading SQL into Team Databases with CloudShell

Overview

Each team's PostgreSQL database is private, so you load SQL from a CloudShell VPC environment placed inside that team's VPC, using the `run-leap-sql.sh` script.

A normal CloudShell session runs outside the team's network and cannot reach the database. The database accepts connections on port 5432 only from the stack's `TasksSecurityGroup`, so you attach that group to the CloudShell environment. No template, security group or Jenkins change is needed.

Plan for about 10 minutes per team the first time. Each team is in a different AWS region, so you create one VPC environment per team.

Before you start

You need an AWS console login that can use CloudShell and create a VPC environment, the `run-leap-sql.sh` script, and each team's `.sql` files.

- **Permissions:** an administrator login has everything needed. Otherwise your user needs CloudShell access plus the EC2 network permissions AWS lists for VPC environments (describing VPCs, subnets and security groups, and creating network interfaces).
- **Stacks:** each team's `leap-<team>` stack must be deployed and healthy, including the database. The template update with the new task definitions does not need to be applied first, but the stack must exist.
- **Database name and user:** the script assumes database `leap` and user `leapadmin`, as in the template. Override with `DB_NAME` and `DB_USER` if a team differs.
- **Rerunnable SQL:** scripts should be safe to run twice (`CREATE TABLE IF NOT EXISTS`, upserts), so a repeat run does not fail or duplicate test data.

Teams, regions and databases

Every team uses the same names with its own team number; only the region differs.

Team	Region	CloudFormation stack	Database instance
team1	us-east-1	leap-team1	leap-team1-postgres
team2	us-east-2	leap-team2	leap-team2-postgres
team3	us-west-1	leap-team3	leap-team3-postgres
team4	us-west-2	leap-team4	leap-team4-postgres
team5	ca-central-1	leap-team5	leap-team5-postgres

Step 1: find the subnet and security group

You need three things from the team's stack: the VPC, one private subnet, and the `TasksSecurityGroup`.

1. In the AWS console, switch to the team's region (top-right region menu).
2. Open **CloudFormation**, select the stack (for example `leap-team1`), and open the **Resources** tab.
3. Note the physical IDs for the logical IDs `PrivateA` (a subnet, `subnet-...`) and `TasksSecurityGroup` (`sg-...`).
4. Open the subnet's ID link; its details page shows the VPC ID (`vpc-...`).

You can also get the first two from any CloudShell session in that region:

```
REGION=ca-central-1; TEAM=team5

aws cloudformation describe-stack-resources --region $REGION --stack-name leap-$TEAM \
  --query 'StackResources[].[LogicalResourceId,PhysicalResourceId]' --output
text \
  | grep -E 'PrivateA|TasksSecurityGroup'
```

Use a **private** subnet (`PrivateA` or `PrivateB`). Private subnets send outbound traffic through the NAT gateway, which is what lets you install the `psql` client.

Step 2: create the CloudShell VPC environment

Create the VPC environment in the same region as the team, with the team's subnet and `TasksSecurityGroup`. Menu wording in the console can differ slightly from what is written here.

1. With the console still in the team's region, open **CloudShell** (the terminal icon in the top bar).
2. Open the **Actions** menu and choose **Create VPC environment**.
3. Enter a name such as `leap-team1-sql`.
4. **VPC:** select the team's VPC from Step 1. (Use the 10.xx.xx.xx VPC)
5. **Subnet:** select the private subnet (`PrivateA`).
6. **Security group:** select the `TasksSecurityGroup` for that stack. Do not pick a different group; the database only trusts this one.
7. Choose **Create**, wait for the new tab to start, and confirm the prompt appears.

A VPC environment has no persistent storage, so files you upload are gone when the session ends. Upload the script and SQL files again each time.

As a quick check, run `curl -s -m 5 https://checkip.amazonaws.com`. It should return the NAT gateway's public address. No answer means the subnet has no outbound route, so use the other private subnet.

Step 3: bring in the script and SQL files

Get `run-leap-sql.sh` and the team's `.sql` files into the CloudShell session by upload or by cloning their repository. Make a temporary S3 buckets and copy your required SQL files and scripts. Then perform the following:

```
mkdir sql && cd sql

aws s3 cp s3://leap-sql-staging-team5/ . --recursive
```

Then make the script executable:

```
chmod +x run-leap-sql.sh
```

If the files are in a directory, name the files so they sort in run order, for example `01_schema.sql`, `02_data.sql`. The script runs a directory's `.sql` files in name order.

Step 4: run the script

Run `./run-leap-sql.sh <team> <files or directory>`; the script finds the region, database and password itself and asks you to confirm before running anything.

```
./run-leap-sql.sh team1 schema.sql data.sql    # specific files, in the order given
./run-leap-sql.sh team3 ./sql/                 # a directory: *.sql in name order
./run-leap-sql.sh -t team5 ./sql/              # each file in one transaction
./run-leap-sql.sh -y team2 ./sql/              # skip the confirmation prompt
```

The script does these things in order:

1. Installs the `psql` client if it is missing (`sudo dnf install postgresql15`).
2. Looks up the database endpoint and the password secret for that team in its region.
3. Shows the team, region, database and file list, and asks `Run these against <team>? [y/N]`.
4. Tests the connection, then runs each file, stopping at the first SQL error.

The password is read from AWS Secrets Manager and is never printed. Check the confirmation screen before you answer `y`: the team and region must match the VPC environment you are in.

Use `-t` for scripts that only create tables and insert rows. Do not use it for scripts that contain `CREATE DATABASE`, `VACUUM` or `CREATE INDEX CONCURRENTLY`; PostgreSQL cannot run those inside a transaction.

Step 5: verify the load

Run a small verification file as the last file in the run; it lists the tables and gives an exact row count for each.

Save this as `99_verify.sql` next to the other files (a directory run applies it last):

Compare the counts with what the team expects. If they are all zero, the scripts created tables but loaded no rows, or they loaded into a different schema than `public`.

The database is shared by the team's four services, so check that table names from different services do not collide.

Troubleshooting

What you see	Likely cause	Fix
"Could not connect to the database" after a pause	CloudShell is not a VPC environment, or it is in the wrong region, subnet or security group	Check the environment's region matches the team. Confirm `TasksSecurityGroup` is attached. Create a new VPC environment if it is not.
"Create VPC environment" is missing or denied	Your login lacks CloudShell or EC2 network permissions	Use an administrator login, or add the EC2 permissions AWS lists for VPC environments.
`sudo dnf install` hangs or fails	The subnet has no outbound route	Create the environment in the other private subnet (`PrivateB`), or check the stack's NAT gateway.
"Could not find the database endpoint or secret"	Wrong team or region, or the stack is not deployed	Check the team, region and stack name against the table above.
`AccessDenied` on `secretsmanager` or `cloudformation`	Your login cannot read those in that region	Use an administrator login and confirm the console region.
`password authentication failed`	Wrong user override, or the secret belongs to another database	Unset `DB_USER` and `DB_NAME`, or set them to the template's values (`leapadmin`, `leap`).
`relation ... already exists`	The SQL is not rerunnable	Add `IF NOT EXISTS`, or drop the tables first.
`CREATE DATABASE cannot run inside a transaction block`	You used `-t`	Run again without `-t`.
Files are gone after reopening CloudShell	VPC environments keep no storage between sessions	Upload the script and SQL files again.

Cleanup and safety

Delete each VPC environment when you finish, because it has the same network access as the team's services.

- **Delete the environment:** in CloudShell, open **Actions**, then **Delete CloudShell environment** (wording may differ). This also removes its network interface in the team's subnet.
- **Master user:** the script connects as `leapadmin`, which can do anything in that database. Read the SQL first for `DROP`, `CREATE DATABASE` and role changes.
- **Passwords:** the script never prints or stores the database password. Do not copy it into notes or chat.
- **Team separation:** each run touches one team's database. Check the confirmation screen for the right team and region before answering `y`.
- **Cost:** CloudShell has no separate charge, but data sent through the stack's NAT gateway is billed as usual.